

Module 2 Assignment: Task Manager

Overview

- **Module:** Module 2: Frontend Development (React Web)
- **Type:** Module Assignment
- **Estimated Time:** 2 hours
- **Submission:** GitHub repository link

Learning Objectives Covered

This assignment reinforces:

- Managing application state with `useReducer` and a well-defined set of actions
- Sharing state and dispatch across the component tree with `useContext`
- Adding client-side routing with React Router

Description

Build a **Task Manager**: a small web app where users can view, add, delete, and filter tasks. State is managed with `useReducer` and shared via `useContext`. The app has two routes: a task list and a task detail page.

This app is intentionally separate from the CRM so you can practise applying Context and Reducer patterns to a new domain without referencing prior lesson code.

What You Will Build

- A task list page at `/tasks` showing all tasks with their status and priority
- A task detail page at `/tasks/:id` showing the full description of a single task
- A filter bar to narrow the list by status
- A form to add new tasks
- A delete button on each task row

Setup

- ☐ Create a new React app: `npm create vite@latest task-manager -- --template react`
- ☐ Install React Router: `npm install react-router-dom`
- ☐ No other libraries are required

Task Data Structure

Each task should have the following shape:

```
{  
  id: 1,
```

```

    title: 'Set up project repository',
    description: 'Initialise a Git repo, add a .gitignore, and push the first
commit.',
    status: 'done',          // 'todo' | 'in-progress' | 'done'
    priority: 'high',        // 'low' | 'medium' | 'high'
  }

```

Seed the app with at least 6 tasks (a mix of statuses and priorities) as the initial state in your reducer.

Core Requirements

1. Reducer

Create a `taskReducer` in `src/reducer/taskReducer.js` that handles these actions:

Action type	Payload	Effect
<code>ADD_TASK</code>	task object	Append to <code>tasks</code>
<code>DELETE_TASK</code>	task <code>id</code>	Remove from <code>tasks</code>
<code>SET_FILTER</code>	'all', 'todo', 'in-progress', or 'done'	Update <code>filter</code>

Initial state:

```

{
  tasks: [ /* your seed data */ ],
  filter: 'all',
}

```

2. TaskContext

Create a `TaskContext` in `src/context/TaskContext.jsx`:

- ☐ Backed by `useReducer` with your `taskReducer`
- ☐ Provide `tasks`, `filteredTasks`, `filter`, `addTask`, `deleteTask`, and `setFilter`
- ☐ `filteredTasks` must be derived during render, not stored in state

3. Routing

Use React Router to create the following routes:

Path	Component	Notes
<code>/tasks</code>	<code>TaskListPage</code>	Shows the list and filter bar
<code>/tasks/:id</code>	<code>TaskDetailPage</code>	Shows the full task description
<code>/</code>		Redirect to <code>/tasks</code>

4. Components

a) **Header**: shows the app name

b) **FilterBar**: four buttons (All / To Do / In Progress / Done); calls `setFilter`; highlights the active filter

c) **TaskList**: renders `filteredTasks`; each row shows the title, status badge, and priority; each row links to `/tasks/:id`; each row has a Delete button that calls `deleteTask`

d) **AddTaskForm**: a controlled form with title (text), description (textarea), status (select), and priority (select); calls `addTask` on submit; clears on success

e) **TaskDetailPage**: reads the task `id` from `useParams`; finds the matching task from `TaskContext`; displays all fields; shows a "Task not found" message if the id does not match any task

5. App Component

- ☐ Wrap the app in `TaskContext.Provider`
- ☐ Set up `BrowserRouter` with the route definitions above
- ☐ Render `Header` on all pages

Example Seed Data

```
const initialTasks = [
  { id: 1, title: 'Set up project repository',      description: 'Initialise
a Git repo, add a .gitignore, and push the first commit.',      status:
'done',          priority: 'high'  },
  { id: 2, title: 'Design database schema',        description: 'Draft the
ERD for the contacts and deals tables.',          status:
'done',          priority: 'high'  },
  { id: 3, title: 'Build login page',              description: 'Create a
login form with email and password fields and basic validation.', status: 'in-progress', priority: 'high'  },
  { id: 4, title: 'Write unit tests for reducer',  description: 'Cover
ADD_TASK, DELETE_TASK, and SET_FILTER with at least two cases each.', status: 'todo',          priority: 'medium' },
  { id: 5, title: 'Update README',                description: 'Add setup
instructions, a screenshot, and a description of the tech stack.', status:
'todo',          priority: 'low'   },
  { id: 6, title: 'Deploy to Vercel',              description: 'Connect the
GitHub repo to Vercel and configure environment variables.',      status:
'todo',          priority: 'medium' },
];
```

Bonus Challenges (Optional)

Easy

- ☐ Show a task count summary above the list: "Showing X of Y tasks"
- ☐ Disable the Add Task form's submit button while any required field is empty

Medium

- ☐ Persist tasks in `localStorage` so they survive a page reload (use a lazy `useState` initialiser or `useEffect` in `TaskContext`)
- ☐ Add an `UPDATE_TASK` action and an inline edit form on the detail page

Hard

- ☐ Add drag-and-drop reordering of tasks in the list using only browser drag events (no library)
- ☐ Add a priority filter on top of the status filter, so both can be active at the same time

Submission

Include a `README.md` in your repository with:

- How to install and run the project
- A screenshot showing the main features working
- A note on any bonus challenges completed

Submit your repository link via NTU Blackboard.

AI and Tools

You may use AI assistants (Claude, Copilot, ChatGPT) to help understand concepts, debug errors, or review your code. However, you must be able to explain every line you submit. In your `README.md`, briefly note:

- Which tools you used and what you used them for
- Any code you adapted from external sources (include the link)

Resources

- [React docs: Extracting State Logic into a Reducer](#)
- [React docs: Passing Data Deeply with Context](#)
- [React docs: Scaling Up with Reducer and Context](#)
- [React Router docs](#)